悲観的開発手法

悲観的開発手法とは

- 対象: テスト基盤・デバッガ・チーム文化がない現場
- レガシーの定義の一つ(Feathers): 変更しやすい・変更後にすぐ確認できる・理解できている、のどれかが欠けたコード
- 問い: その現場で、品質をどう守るか
- 悲観的: **最悪を先に想定して備える姿勢**(この手法での定義)

https://www.informit.com/store/working-effectively-with-legacy-code-9780132931779

4本の柱をまとめる総称が、悲観的開発手法

1. 悲観的設計
2. 悲観的TDD(最初の名前)
3. 悲観的チーム実践(実践の方法を作成済み)
4. 悲観的メンタリング(カリキュラムを作成済み)

TDD: 失敗する自動テストを先に書いてから実装する開発(Beck)

https://www.informit.com/store/test-driven-development-by-example-9780321146533

13章の文書にまとめている(主な内容)

- 3つの基本原則
  - 【差し込み: 原則の名前と1行説明を3つ】
- 6つの技法カテゴリ
- レガシー度合いのスペクトラム
- LLMの活用(「AI仮実行」という技法を含む)
- 心理面の枠組み

https://platform.claude.com/docs/en/about-claude/glossary

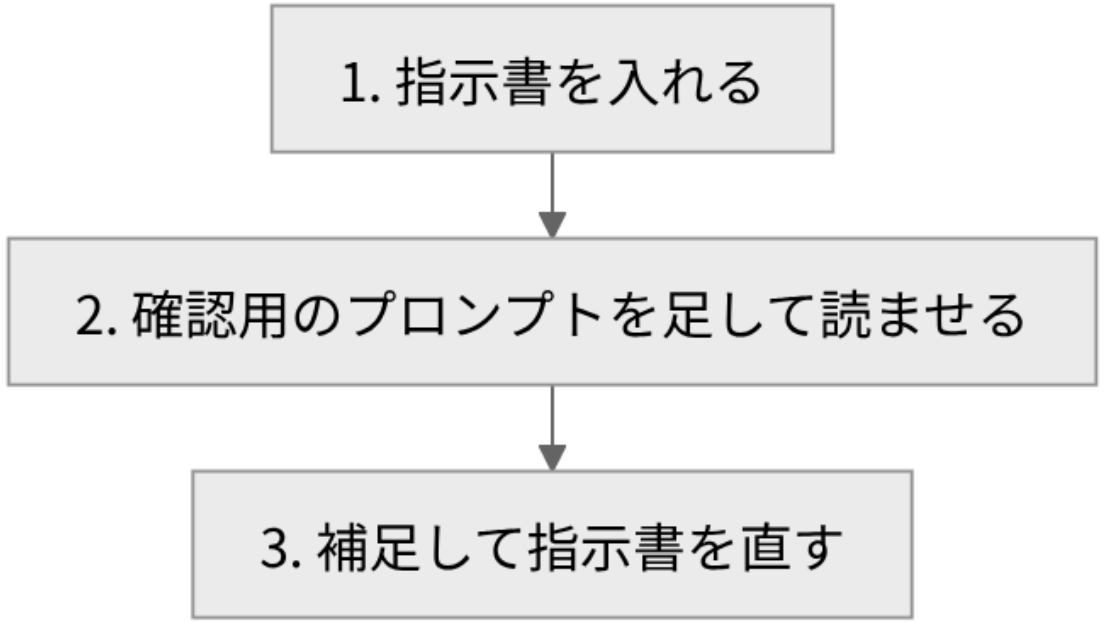
「書いたつもり」を疑う技法

- 暗黙知: 言葉にしきれない知識(Polanyi)
- 記事での意味: 頭にあるが、指示書に書いていない前提

	賢いモデル	**小型モデル(Haiku)**
曖昧な指示	文脈から補う	補えない
書き手から見ると	書いたつもりになる	推測として表に出る

記事の主張: リベシティ ノウハウ図書館(2026/07/13)

https://library.libecity.com/articles/01KXB75HC6S3NSE7MM7SSGJ2J0
https://press.uchicago.edu/ucp/books/book/chicago/T/bo6035368.html



- 読ませる相手: 小型モデルのサブエージェント
- 説明させる内容: 作られるもの・作業の手順・利用者
- 条件: 質問は禁止、推測した箇所は「推測」と明記

https://library.libecity.com/articles/01KXB75HC6S3NSE7MM7SSGJ2J0
https://code.claude.com/docs/en/sub-agents

- 公式で確認(2026年10月時点)
  - サブエージェントは model に haiku を指定できる
  - 会話履歴は引き継がないが、CLAUDE.md は受け取る
  - CLAUDE.md を外す設定が omitClaudeMd
  - Haikuは小型で速く、低コスト(最新は5.5)
- 未検証: 小型モデルで暗黙知が見つかるか(まだ試していない)
- 検討中: 悲観的開発手法への取り入れ

【差し込み: 取り入れ先の候補と、AI仮実行との関係】
【差し込み: 試した結果(試した場合のみ)】

https://code.claude.com/docs/en/sub-agents
https://www.anthropic.com/claude-haiku-5-5

まとめ

- 悲観的開発手法は、テスト基盤・デバッガ・チーム文化がない現場の方法論
- 柱は4本: 設計 / TDD / チーム実践 / メンタリング
- 暗黙知は、読み手が補えない箇所に表れる(記事の主張)
- 小型モデルに指示書を読ませる手法は未検証で、取り入れは検討中